



EE5114: Autonomous Robot Navigation

Continuous Assessment 2

Name: Lu Zixuan

Student ID: A0285204J

15/October/2024

Introduction

The aim of this project is to implement Fast Slam algorithm for a UAV, enabling it to accurately estimate its position and map the environment based on sensor data.

This report will detail the completed sections of code accompanied by snapshots of the relevant code and explanations of the formulas utilized. The original algorithm for SLAM are also analysed, such as the logic behind the line-fitting algorithm and the rationale for constraining changes in position and orientation in motion estimation. The final results obtained from running the `motion_estimation.m` and `slam.m` are illustrated in the figures below.

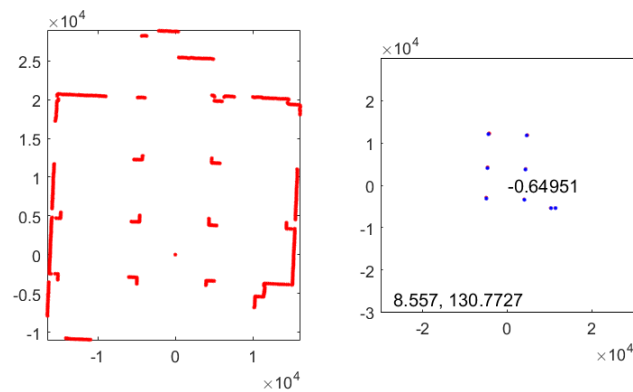


Figure 1: LiDAR Data Points and Motion Estimation

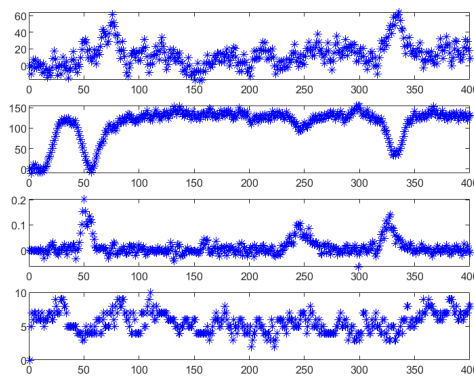


Figure 2: Associated Points Count Over Time

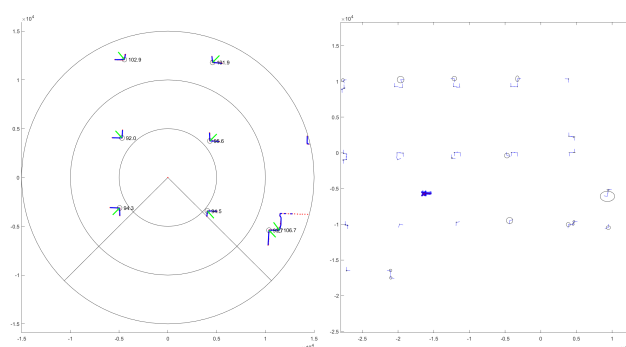


Figure 3: Feature Map from LiDAR Data

Questions and Solutions

1. Please explain all your filled-in codes (6 parts in total) with a snapshot of those lines of code. List formulas you have used for that part of codes if applicable.

1.1 slam_lidar_split_merge.m

This section of the code computes the perpendicular distance from any point (x, y) in the set to the line segment formed between the first point (x_1, y_1) and the last point (x_2, y_2) . This distance helps determine how well the line fits the set of points. If the distance of the farthest point exceeds a predefined threshold, the set will be split into two subsets, and the process is repeated recursively. The distance d from a point to the line can be calculated using the following:

$$d = \frac{|(x_2 - x_1)(y_1 - y) - (x_1 - x)(y_2 - y_1)|}{\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}} \quad (1)$$

```
slam_lidar_split_merge.m (line 43-61)
    % Missing codes start here ...

    % Find furthest point to this line
    point_distances = zeros(1, last);
    winner_value = abs(point_distances(1));
    winner_index = 1;

    for i = 1:last
        %Calculate the distance of each point to the estimated line
        N = (x2-x1) * (y1 - points(i).y) - (x1 - points(i).x) * (y2-y1);
        D = sqrt((x2 - x1)^2 + (y2 - y1)^2);
        point_distances(i) = abs(N/D);

        %Check if this point is farther than the current winner
        if point_distances(i) > winner_value
            winner_value = point_distances(i);
            winner_index = i;
        end
    end

    % Missing codes end here ...
```

1.2 slam_lidar_feat_extrn.m

This section of the code computes the angle between two consecutive line segments and determines the direction of the corner point formed by these segments. Since the two segments are directional, the angle is computed using the dot product formula, followed by the calculation of the corner's orientation.

$$\cos(\theta) = \frac{\mathbf{v}_a \cdot \mathbf{v}_b}{\|\mathbf{v}_a\| \|\mathbf{v}_b\|} \quad (2)$$

$$\theta = \cos^{-1} \left(\frac{\mathbf{v}_a \cdot \mathbf{v}_b}{\|\mathbf{v}_a\| \|\mathbf{v}_b\|} \right) \quad (3)$$

Once the angle is determined, the direction of the corner formed by the two vectors is calculated by adding the unit vectors of both line segments.

$$\mathbf{v}_{\text{heading}} = \frac{\mathbf{v}_a}{\|\mathbf{v}_a\|} + \frac{\mathbf{v}_b}{\|\mathbf{v}_b\|} \quad (4)$$

$$\text{heading} = \text{atan2}(\mathbf{v}_{\text{heading}}(2), \mathbf{v}_{\text{heading}}(1)) \quad (5)$$

```
slam_lidar_featextrn.m (line 27-46)
% Missing codes start here ...

% Calculate angle span of the corner
va = [lines(i+1).p2.x - lines(i).p2.x,
      lines(i+1).p2.y - lines(i).p2.y];
vb = [lines(i).p1.x - lines(i+1).p1.x,
      lines(i).p1.y - lines(i+1).p1.y];

cos_angle = dot(va, vb) / (norm(va) * norm(vb));
angle = slam_in_pi(acos(cos_angle));

% Only use corner features having angle from 60 to 120 degrees
if (angle > deg2rad(60) && angle < deg2rad(120))
    % Calculate heading direction of the corner
    va_unit = va / norm(va);
    vb_unit = vb / norm(vb);
    heading_vec = va_unit + vb_unit;
    heading = slam_in_pi(atan2(heading_vec(2), heading_vec(1)));

% Missing codes end here ...
```

1.3 motion.m

This section of the code implements motion estimation between two sets of corresponding points P and Q by calculating the optimal rotation and translation using Singular Value Decomposition (SVD).

1. Identify corresponding corner features between two frames using Mahalanobis distance.
2. Arrange corner features so that p_i in the previous frame corresponds to q_i in the current.
3. Calculate the centroid: Compute the centroid for both frames, denoted as $\bar{p}$ and $\bar{q}$.
4. Form matrices: Generate P and Q by subtracting the centroids from the feature points:

$$P = [p_1 - \bar{p}, \dots, p_{\max} - \bar{p}] \quad (6)$$

$$Q = [q_1 - \bar{q}, \dots, q_{\max} - \bar{q}] \quad (7)$$

5. Apply SVD: Compute the Singular Value Decomposition (SVD) of the product of PQ^T :

$$[U, S, V] = \text{SVD}(PQ^T) \quad (8)$$

where S is the diagonal matrix of singular values, U and V are orthogonal matrices.

$$d = \text{sign}(\det(VU^T)) \quad (9)$$

6. Calculate the rotation matrix R The rotation matrix R is calculated as follows:

$$R = V \begin{bmatrix} 1 & 0 \\ 0 & d \end{bmatrix} U^T \quad (10)$$

The translation vector T is then calculated by:

$$T = R\bar{p} - \bar{q} \quad (11)$$

7. Final orientation and position changes:

$$c_t = -\text{atan2}(R(2), R(1)) \quad (12)$$

$$\Delta \bar{x}_t = T(1), \quad \Delta \bar{y}_t = T(2) \quad (13)$$

```

motion-estimation.m (line 103-124)
% Missing codes start here ...
% Compute the centroids of the corresponding points P and Q
P_bar = mean(P, 1);
Q_bar = mean(Q, 1);

% Subtract the centroids from the point sets
P_centered = P - P_bar;
Q_centered = Q - Q_bar;

% Perform Singular Value Decomposition (SVD)
[U, S, V] = svd(P_centered' * Q_centered);
d = sign(det(V * U'));

% Compute the rotation matrix R and translation vector T
R = V * [1, 0; 0, d] * U';
T = R * P_bar' - Q_bar';

% Final orientation and position changes:
theta = -atan2(R(2), R(1));
translation = T(1:2);

% Missing codes end here ...

```

1.4 slam_resample.m

This section of the code performs particle resampling as part of a particle filter algorithm. It begins by calculating the total weight of all particles. Then, it creates a cumulative distribution of the normalized particle weights. For each particle, a random number between 0 and 1 is generated to select a particle based on this cumulative distribution, which ensures that particles with higher weights are more likely to be resampled. After resampling, all new particles are assigned the same initial weight.

```
slam_resample.m
function [particles] = slam_resample(particles, init_weight)
    particles_count = size(particles, 2);
    new_particles = particles;

    weights = zeros(particles_count, 1);
    weight_total = 0;

    for i = 1:particles_count
        weights(i) = particles(i).weight;
        weight_total = weight_total + weights(i);
    end

    % Calculate cumulative weights and normalize them
    cumulative_weights = cumsum(weights) / weight_total;

    for i = 1:particles_count
        % Missing codes start here

        % Resamples particles based on their weights
        random_num = rand(1);
        index = find(cumulative_weights >= random_num, 1);
        new_particles(i) = particles(index);

        % Afterwards, each new particle should be given the same
        ↪ init_weight
        new_particles(i).weight = init_weight;

        % Missing codes end here
    end

    particles = new_particles;
end
```

1.5 slam_crn timer_kf.m

This code performs the Kalman Filter update for a detected corner in SLAM, updating the corner's state (position and orientation) and covariance (uncertainty) based on new observations. The Kalman gain combines prior estimates with new measurements to minimize error, updating the state and reducing uncertainty.

$$\begin{aligned} K &= P_{k|k} Q^{-1}; Q = P_{k|k} + Q_t \\ \text{innovation} &= z_k - H \hat{x}_{k|k} \end{aligned} \quad (14)$$

Based on the Kalman gain K and the innovation, the state of the known corner is updated using the following formulas:

$$\hat{x}_{k|k+1} = \hat{x}_{k|k} + K \cdot \text{innovation} \quad (15)$$

The covariance update formula describes how the new measurements affect the covariance of the known corner:

$$P_{k+1|k+1} = (I - KH)P_{k|k} \quad (16)$$

$$P_{\text{new}} = (I - K) \cdot P_{\text{old}} \quad (17)$$

Where:

- P_{new} is the updated covariance matrix.
- I is the identity matrix with the same dimensions as P .

```
slam_crn timer_kf.m (line 16-27)
% Missing codes start here ...
% Update mean of this corner
current_state = [known_corner.x; known_corner.y;
    ↪ known_corner.heading; known_corner.angle];
updated_state = current_state + K * innovation;
known_corner.x = updated_state(1);
known_corner.y = updated_state(2);
known_corner.heading = updated_state(3);
known_corner.angle = updated_state(4);

% Update covariance of this corner
I = eye(size(known_corner.covariance)); % Identity matrix
known_corner.covariance = (I - K) * known_corner.covariance;

% Missing codes end here ...
```

1.6 slam.m

This section of the code updates each particle's state in a particle filter. It retrieves motion estimate and adjusts the particles' positions and orientations based on these noisy estimates.

$$\begin{aligned}\Delta c_t^{[m]} &= \Delta \bar{c}_t(1 + \alpha_1 \text{randN}(1)) + \alpha_2 \text{randN}(1), \\ \Delta x_t^{[m]} &= \Delta \bar{x}_t(1 + \alpha_3 \text{randN}(1)) + \alpha_4 \text{randN}(1), \\ \Delta y_t^{[m]} &= \Delta \bar{y}_t(1 + \alpha_3 \text{randN}(1)) + \alpha_4 \text{randN}(1).\end{aligned}\tag{18}$$

- α_1 represents theta_noise_proportion and α_2 represents theta_noise_add.
- α_3 represents translation_noise_proportion and α_4 represents translation_noise_add.

$$\begin{aligned}P_t^{[m]}.c &= P_t^{[m]}.c + \Delta c_t^{[m]}, \\ P_t^{[m]}.x &= P_t^{[m]}.x + \cos(P_t^{[m]}.c) \Delta x_t^{[m]} - \sin(P_t^{[m]}.c) \Delta y_t^{[m]}, \\ P_t^{[m]}.y &= P_t^{[m]}.y + \sin(P_t^{[m]}.c) \Delta x_t^{[m]} + \cos(P_t^{[m]}.c) \Delta y_t^{[m]}. \end{aligned}\tag{19}$$

```
slam.m (line 74-93)
% Missing code start here ...

% Retrieve motion estimates for the current iteration 'i'
dx = motion_estimate(i).x;
dy = motion_estimate(i).y;
dtheta = motion_estimate(i).theta;

% Apply rotation to particles with (additive and proportional) noises
noisy_dtheta = dtheta * (1 + theta_noise_proportion * randn(1)) +
    ↪ theta_noise_add * randn(1);

% Apply translation to particles with (additive and proportional)
    ↪ noises
noisy_dx = dx * (1 + translation_noise_proportion * randn(1)) +
    ↪ translation_noise_add * randn(1);
noisy_dy = dy * (1 + translation_noise_proportion * randn(1)) +
    ↪ translation_noise_add * randn(1);

% Update particle positions based noisy estimates and updated theta
particles(j).theta = slam_in_pi(particles(j).theta + noisy_dtheta);
particles(j).x = particles(j).x + noisy_dx * cos(particles(j).theta)
    ↪ - noisy_dy * sin(particles(j).theta);
particles(j).y = particles(j).y + noisy_dx * sin(particles(j).theta)
    ↪ + noisy_dy * cos(particles(j).theta);

% Missing code end here ...
```


2. Please explain what problem is line 11-40 in the original ‘slam_lidar_split_merge.m’ trying to solve? Why does the solution need to use if/else to consider two cases?

Lines 11-40 in the script aims to calculate the parameters of the line equation that best fits a line segment determined by two endpoints. The problem it addresses is the conversion of a segment defined in Cartesian coordinates to a line representation in polar coordinates. The line parameters being calculated are r and θ , which correspond to the polar coordinates of the line.

The reason for using the if/else is to handle different geometric orientations ($y = mx + c$ or $x = my + c$) of the line segment. The structure allows the code to distinguish between line segments that are primarily horizontal versus those that are primarily vertical. In this case, it effectively avoids the situation where the denominator becomes infinitely small during slope calculation to ensure stability and accuracy.

3. Please explain what line 51-97 in ‘motion_estimation.m’ is trying to do?

Lines 51-97 are dedicated to matching detected feature points and corners between two consecutive frames using Mahalanobis distance. The code begins by checking if corners are detected in both the current (`corners2`) and previous (`corners1`) frames. If both are non-empty, it proceeds by calculating the number of corners in each frame and initializing a `mahalanobis_matrix` with values set to a predefined threshold. Nested loops then compute the Mahalanobis distance between each pair of corners based on their x , y coordinates and angles, factoring in the inverse covariance matrix. If the distance is below the threshold, the matrix is updated.

The code then iteratively matches corners by finding the minimum distance in the matrix, confirming matches if the value is under the threshold, and recording them in arrays `P` and `Q`. Matched corners are removed from further consideration by updating their matrix rows and columns. The loop ends when no values remain below the threshold. This part of the script enables the tracking of feature points across frames based on spatial and orientation consistency.

4. Please explain what ‘slam_in_pi.m’ function is trying to achieve. Where are the locations this function is called. Why are they necessary?

The function ‘slam_in_pi.m’ is designed to restrict the angle to lie within the range $[-\pi, \pi]$ radians. As the heading or orientation can be represented by angles that could accumulate over time and extend beyond a full circle.

In CA2 this function is called in the followings: `slam_lidar_split_merge.m`, `slam_lidar_feat_ex-trn.m`, `slam_cnr_loc2glo.m`, `slam_cnr_kf.m`, `slam_cnr_jcbb_assoc.m`, `slam.m`, `motion_estimate.m`. This function appears when it comes to calculating angles, such as headings or theta.

This function is necessary as angles are periodic, an angle of 2π radians (360°) is the same as 0 radians (0°), and larger angles like 4π also correspond to 0 radians. In slam, it’s important to express the UAV’s heading angle within the fixed range of $[-\pi, \pi]$ to avoid ambiguity and maintain consistency. If the angle is not restricted to this range, it can accumulate indefinitely as the UAV turns, making it be incorrectly identified and difficult to use for further calculations.

5. In the last part of ‘motion_estimation.m’ (before plotting), what do you think is the advantage of constraining the change of x , y and θ ?

The advantage of constraining the changes in x , y , and θ within thresholds is to ensure the motion estimates between consecutive frames are smooth and physically reasonable. the change in x and y is constrained to a maximum of 10 units, while the change in θ is constrained to 0.05 radians. Without these constraints, sudden or large changes in translation or rotation could occur due to noise in sensor data or other uncertainties in the environment. These unrealistic changes can lead to incorrect motion estimation, causing unstable behavior in the system.

```
motion_estimation.m (line 126-146)
% Constrain x change to be within a threshold
if (translation(1) - translation_pre(1) > 10)
    translation(1) = translation_pre(1) + 10;
elseif (translation(1) - translation_pre(1) < -10)
    translation(1) = translation_pre(1) - 10;
end

% Constrain y change to be within a threshold
if (translation(2) - translation_pre(2) > 10)
    translation(2) = translation_pre(2) + 10;
elseif (translation(2) - translation_pre(2) < -10)
    translation(2) = translation_pre(2) - 10;
end

% Constrain theta change to be within a threshold
if (slam_in_pi(theta - theta_pre) > 0.05)
    theta = theta_pre + 0.05;
elseif (slam_in_pi(theta - theta_pre) < -0.05)
    theta = theta_pre - 0.05;
end
theta = slam_in_pi(theta);
```